

LG-CUBESAT · FLIGHT SOFTWARE V1.1

System & Compute Guide

How the OBC architecture works, and how to review the async compression engine built for `compute` — with a suggested reading order.

Covers: 7-process OBC architecture, IPC/CSP protocols, the `compute` process (adaptive Golomb–Rice compression, async dispatch, cancellation), and the test suite.

Status at time of writing: all 7 OBC processes real and working; 7/7 automated tests passing on a clean build.

Reading Order

This is the order to actually open files in. It follows the dependency chain — each file builds on the ones before it — rather than alphabetical or directory order. Skip nothing on a first pass; each step is short.

Part A — Orient yourself (the spec, not the code)

1. `apps/obc/roles.md`

What each of the 7 OBC processes is *for*. Read this before any code — it's the contract every process below implements.

2. `apps/obc/threading.md`

How each process is internally structured (reactive thread vs. periodic thread). `compute`'s row here is the exact spec its dispatcher/worker design follows.

3. `apps/obc/pthreads.md`

The mechanical convention every thread in this codebase follows: an `X_thread_init()` / `X_thread()` pair. Once you've seen this, every process's `main.c` reads the same way.

Part B — The two transports everything rides on

4. `apps/obc/ipc/include/obc_ipc.h`

The internal, process-to-process transport. Every OBC role uses this. Note `MAX_IPC_PAYLOAD` (256 bytes) in the `.c` file — it's the reason several protocols below are "chunked."

5. `apps/obc/ipc/src/obc_ipc.c`

Skim only. One thing matters: `IPC_send` is deliberately fail-fast (see its own comment) — it never retries a failed `connect()`. This single design choice explains two real bugs described later in this document.

6. `shared/csp/csp_commands.h`

The *external* wire contract — CSP addresses, ports, and command structs used to talk to ADCS/EPS over the simulated I2C bus. Internal-only features (like `compute`) don't touch this at all.

Part C — The already-working processes (read fast; you built these)

7. `apps/obc/supervisor/src/processes.c`

Process lifecycle: spawn, reap, restart.

8. `apps/obc/fdir/src/health_monitor.c` + `fallback.c`

Fault policy: tracks board resets, decides on shutdown.

9. `apps/obc/commands/src/ingest.c` + `relay.c`

The only process that touches the external CSP bus.

10. `apps/obc/time/src/time_sync.c`

Simplest complete example of the request/reply-over-IPC pattern.

11. **apps/obc/mission/src/scheduler.c + payload_commander.c**

The balloon timeline; `payload_commander_compress_photo()` here is the client-side half of the new compute feature.

Part D — data : the piece that makes compute possible

12. **apps/obc/include/obc_data_protocol.h**

The read *and* write chunk protocol. Read this before `compute` — its worker thread is built entirely on top of this contract.

13. **apps/obc/data/src/filesystem.c**

The only file in the whole codebase that calls `fopen`. Note `ipc_send_retrying` near the top — a real bug fix explained in Part F.

14. **apps/obc/data/src/storage.c**

Dispatches an inbound IPC message to the read or write path purely by its byte size.

Part E — compute itself: read in this exact order

15. **apps/obc/compute/include/bitstream.h + src/bitstream.c**

The foundation. A bit-level writer/reader — nothing above this file makes sense without it.

16. **apps/obc/compute/include/rice_codec.h + src/rice_codec.c**

The actual compression algorithm. This is the dense one — Part 3 of this document walks through it concept-by-concept before you read the code.

17. **apps/obc/include/obc_compute_protocol.h**

The IPC wire contract for compute: `compute_compress_request_t`, `compute_cancel_request_t`, `compute_result_t`.

18. **apps/obc/compute/src/compute.c**

The dispatcher/worker/mailbox engine. Read this last — it ties `bitstream`, `rice_codec`, and the data protocol together into the actual running process.

Part F — Prove it to yourself

19. **tests/test_rice_codec.c**

Direct calls into the codec, no processes. Fastest way to see the algorithm's edge cases.

20. **tests/test_compute_async.c**

Proves BUSY-rejection and cancellation against real running processes.

21. **tests/test_full_constellation.c**

Proves the whole 7-process system works together, not just in isolation.

Total: ~21 files, most of them short. Budget 60–90 minutes for a careful first pass. Everything from here on in this document explains *why* each of those files looks the way it does.

Contents

Part 1 — System Overview

- 1.1 The big picture
- 1.2 The 7 OBC processes at a glance
- 1.3 Patterns repeated across every process

Part 2 — The Processes You Already Know

Part 3 — Deep Dive: `compute`

- 3.1 Why compute is different: genuine asynchrony
- 3.2 The dispatcher / worker / mailbox design
- 3.3 The compression algorithm, conceptually
- 3.4 Reading `rice_codec.c`
- 3.5 Real bugs found — a case study in testing

Part 4 — Protocol Reference

Part 5 — Running and Testing the System

Part 6 — Known Gaps

Part 7 — Glossary

Part 1 — System Overview

1.1 The big picture

This repository is flight software for a CubeSat, currently scoped for a November high-altitude balloon test flight — the near-term goal is proving the communication architecture end-to-end before it carries the full satellite mission.

Two kinds of computer exist in this system:

ADCS	A single FreeRTOS application running on real (or simulated) MCU hardware. Reference-quality and complete: command handling, telemetry, a local watchdog, and a full CSP round-trip with the OBC.
OBC	Not one program — 7 independent Linux processes that together form the On-Board Computer. This is the part of the system this document is mostly about.

Every board (OBC, ADCS, and — eventually — EPS, Thermals, Camera, Comms) is a node on a shared **CSP** (CubeSat Space Protocol) bus, simulated today over Unix domain sockets and swappable for a real I2C peripheral later without changing any application code. Inside the OBC, the 7 processes additionally talk to *each other* over a second, completely separate transport: an internal, role-addressed IPC library. Confusing these two transports is the single easiest way to misread this codebase — Part 4 draws the line precisely.

1.2 The 7 OBC processes at a glance

Process	Job	Status
supervisor	Starts, monitors, and restarts the other 6. Mechanism only — no fault judgment.	real
fdir	Fault policy. Tracks board resets, decides when a board needs a full shutdown.	real
commands	The only process allowed to touch the external CSP bus. Routes inbound traffic in, outbound traffic out.	real
mission	Owns the payload/mission timeline and any future mission autonomy.	real
time	Keeps every board's clock aligned to the OBC's.	real
data	Owns <i>all</i> filesystem access — the only process that ever calls <code>fopen</code> .	real

compute	Heavy, potentially slow computation — today, lossless data compression — run asynchronously with cancellation support.	real
---------	--	------

All 7 are real as of this document. `compute` was the last to be finished and is the subject of Part 3.

1.3 Patterns repeated across every process

Once you recognize these, every file in Part C of the reading order looks the same shape:

The init/thread pair

```
int    X_thread_init(void); // creates the thread, one-time setup, returns immediately
void * X_thread(void *arg); // the actual loop body: reactive (blocks on IPC_receive)
                                // or periodic (sleeps to an absolute wake time)
```

The heartbeat

Every long-lived role runs a periodic thread that does nothing but `IPC_send(ROLE_SUPERVISOR, NULL, 0)` once a second. Without it, `supervisor` cannot tell "quietly working" from "hung," and will kill and restart a perfectly healthy process. This bit the project for real once — see the `autonomy_thread` story in Part 4.

Table-driven scalability

Anywhere a process needs to handle "one of several boards" or "one of several periodic actions," the code is a small array plus a lookup, not a chain of `if` statements — `ingest.c`'s port routing table, `time_sync.c`'s board table, `autonomy.c`'s action table. Adding a new board or action is one new row, not new logic.

Sim vs. real platform split

Anything that touches actual hardware (the comms bus, a board reset, a camera, a radio) is declared once as an interface in `shared/interfaces/`, with a `platform/sim/drivers/` implementation used today and a `platform/real/drivers/` implementation filled in — or honestly stubbed — later. Nothing above that interface layer needs to know which one is linked in.

Part 2 — The Processes You Already Know

Brief, since these were built earlier and you already understand them — this is here for completeness and so this document stands alone.

supervisor

`processes.c` resolves each sibling process's path relative to its own executable (so launch directory never matters), spawns all 6 with `posix_spawn`, and periodically reaps exit codes and sweeps for a missed heartbeat. A frozen or crashed process gets `SIGTERM`, a grace period, then `SIGKILL`, then a restart. `supervisor` never decides *whether* a fault matters — that's `fdir`'s job.

fdir

`health_monitor.c` receives fire-and-forget reset notices from external boards — a board decides and acts on a local fault entirely on its own (see ADCS's `fault_manager.c`), then tells the OBC after the fact. Past a threshold of repeated resets, `fallback.c` asks `commands` to relay a shutdown command over CSP.

commands

The sole owner of the external bus. `ingest_thread` binds every inbound CSP port and forwards decoded payloads to whichever internal role owns them, purely by a routing table keyed on port number. `relay_thread` does the reverse: any internal role can ask it to forward an arbitrary payload out over CSP via a generic `relay_request_t` envelope.

mission

`scheduler.c` runs the balloon flight as a small one-shot state machine: wait for ascent → take photo → **compress** → downlink → done. `payload_commander.c` is what actually does each step, including the new `payload_commander_compress_photo()` that talks to `compute`. `autonomy.c` is a separate, recurring table-driven engine for real mission behavior (e.g. periodically pointing ADCS at the sun) — built now, ahead of need, because its shape matters more for the real satellite mission than for this balloon flight.

time

Reuses the same command-envelope/ACK mechanism every board's command handler already implements, rather than inventing a new CSP port: a `CMD_TIME_SYNC` command carries a timestamp to

each board's existing command port. One thread broadcasts on a timer; a second answers an on-demand resync request immediately.

data

The newest piece before `compute`, and the one that makes it possible. `filesystem.c` is the only file left in this codebase that calls `fopen`. Because a single internal IPC message caps at 256 bytes (see `obc_ipc.c`), files cross the read and write paths in chunks — `data_read_reply_t` and `data_write_chunk_t` in `obc_data_protocol.h` — not as one message. `storage.c` tells the two apart purely by the size of the incoming message.

Part 3 — Deep Dive: `compute`

This is the newest and most involved piece of the system, so this section goes slower than Part 2. If you only read one part of this document carefully, make it this one.

3.1 Why `compute` is different: genuine asynchrony

Every other OBC role's IPC handling follows one shape: block on `IPC_receive`, act, maybe reply, loop. That works because nothing else those roles do takes long enough to matter. `compute` breaks that assumption — a compression job might legitimately take a while, and per `apps/obc/roles.md`, "a long-running job [must] never block [the process] from accepting a new request or a cancellation."

`apps/obc/threading.md` specifies the exact shape to use, and `compute.c` follows it directly:

"Two threads: a dispatcher blocks on ipc for work requests; a separate worker thread does the actual compute... so a long job never blocks the dispatcher from accepting a new request or a cancellation."

3.2 The dispatcher / worker / mailbox design

The dispatcher is the only thread that ever calls `IPC_receive`. On a `compute_compress_request_t`: if a job is already running, it replies `COMPUTE_STATUS_BUSY` immediately — there is exactly one job slot, not a queue, because the balloon flight only ever needs to compress one photo at a time, and a queue nobody needs yet is speculative complexity. Otherwise it marks the slot busy and spawns a **detached worker thread** for that job, then goes straight back to listening. On a `compute_cancel_request_t` matching the running job, it just sets a flag — nothing replies to a cancel.

DESIGN SUBTLETY WORTH UNDERSTANDING

`obc_ipc` only supports *one* blocking receiver per process — it's built around a single listening socket. `compute` is the first process where a second thread (the worker) also needs to receive messages (replies from `data`, as it reads and writes file chunks). Having the worker call `IPC_receive` directly would race the dispatcher's own `accept()` loop for the same incoming connections — a message meant for the worker could get handed to the dispatcher instead, or vice versa, unpredictably. The fix: the dispatcher stays the sole receiver, and anything it gets that isn't a request it recognizes gets routed to the worker through a tiny single-slot **mailbox** (a mutex + condition variable). This is why `compute.c`'s `dispatch_thread` checks the sender's role first — a message from `ROLE_DATA` goes to the mailbox; anything else is a request.

The worker's job, once started:

1. Ask `data` to stream the input file back, chunk by chunk, reassembling it into a static buffer (checking the cancellation flag between chunks).
2. Run `rice_compress` on the assembled bytes.
3. Send the compressed result to `data` to write, one chunk at a time, waiting for an acknowledgment before sending the next (checking cancellation between chunks here too).
4. Send a `compute_result_t` back to whoever asked, and clear the job slot.

3.3 The compression algorithm, conceptually

The scheme is **adaptive block Golomb–Rice coding with a zero-block and verbatim fallback** — the same core technique **CCSDS 121.0** ("Lossless Data Compression") uses for real spacecraft telemetry, scoped down to the parts that matter here rather than the entire standard. It is deliberately *not* image-specific: CCSDS 121.0 itself is a general-purpose compressor for any fixed-width sample stream — telemetry, sensor data, or image bytes are all just "a stream of numbers with a native width" to it. (True image-aware compression, CCSDS 122.0, adds a 2D wavelet transform in front of an entropy coder like this one — a separate, larger piece of work, out of scope here.)

Step 1 — Delta (the predictor)

Raw sample values rarely compress well directly. Adjacent samples in real sensor or image data, however, are usually close to each other in value. So the first step replaces every sample with the *difference* from the previous one — small, since most neighboring values are similar.

THE SUBTLETY THAT CAUSED A REAL BUG

This difference must be computed as **unsigned, wraparound arithmetic at the sample's own bit width** — not a wider type. For an 8-bit sample, `106 - 247` must wrap to `115` (i.e. `mod 256`), not become some large 32-bit negative-looking number. Decoding reverses it with the matching wraparound addition — addition and subtraction $\text{mod } 2^n$ are always exact inverses, regardless of the individual numbers' true magnitude. Get the width of this wraparound wrong (as an early version of this code did — it wrapped at 32 bits unconditionally) and everything after it silently corrupts for any narrower sample width. See 3.5 for the full story.

Step 2 — Zig-zag mapping

Rice coding (next step) only works on non-negative integers, but a delta can be negative. Zig-zag mapping interleaves signed values onto the non-negative number line so small magnitudes — positive *or* negative — both map to small unsigned numbers:

```
0, -1, 1, -2, 2, -3, 3, ... -> 0, 1, 2, 3, 4, 5, 6, ...
```

Step 3 — Rice coding, per 16-sample block

For a non-negative integer `n` and a chosen parameter `k`, split it into a quotient and remainder:

```
q = n >> k          (written in unary: q one-bits, then a terminating zero-bit)
r = n & ((1<<k)-1)  (written as a fixed k-bit field)
```

Unary coding is wildly inefficient for large numbers but nearly free for small ones — exactly the bet that pays off once most values are small post-delta. `k` is chosen per 16-sample block by trying a small range and keeping whichever produces the fewest total bits, so the codec adapts to each block's local statistics rather than using one global setting.

Step 4 — Two more options per block, for robustness

Block type	When used	What's stored
Zero-block (the RLE case)	Every mapped value in the block is exactly 0 — a uniform run.	A 2-bit flag. Nothing else.
Rice-coded	The general case.	2-bit flag + 5-bit <code>k</code> + each value's Rice code.
Verbatim	Even the best <code>k</code> would produce <i>more</i> bits than just storing the block raw (high-entropy/random data).	2-bit flag + each value at its full sample width, uncompressed.

The verbatim option is what makes this a genuine, safe general-purpose compressor rather than something that can blow up arbitrarily on hostile input — the worst case is always bounded near "no compression," never "much larger than the input."

Why sample width is a real parameter

Delta-ing a multi-byte sample stream one byte at a time produces garbage, not small residuals — the width has to match the data's real structure. So the whole codec is parameterized by width (1, 2, or 4 bytes), making it equally correct for today's mock photo (width 1: raw bytes) and a future fixed-width telemetry log (width 4: e.g. logged `int32_t` readings) with no protocol change, just a different number.

3.4 Reading `rice_codec.c`

With the concepts above in hand, the file itself is short. Read it in this order:

`zigzag_encode`

Reinterprets a wraparound value as signed at the right width, then applies the zig-zag formula.

`zigzag_decode`

The exact inverse.

`max_k_for`

Caps the search range at 31, not 32 — shifting a 32-bit value by a full 32 bits is undefined behavior in C, even though "k=32" is what the math would otherwise suggest.

`rice_compress`

Header, then delta+zigzag every sample up front into a scratch array, then the per-block loop (try zero → try best `k` vs. verbatim → write whichever wins).

`rice_decompress`

The exact mirror: read the header, then per block read the type, then either infer zeros, Rice-decode, or read verbatim values.

The stream header (`n_samples` , `tail_len` , `width`) is plain bytes, not bit-packed — it exists so a byte count that isn't an exact multiple of the sample width still round-trips correctly (the leftover bytes are stored raw as a "tail"), and so decode can sanity-check the width it was called with against what encode actually used.

3.5 Real bugs found — a case study in testing

Five real, load-bearing bugs were found while building this feature — every one of them by actually running code and observing wrong behavior, not by reading it. This section is worth reading even if you skip everything else, because it's the strongest argument in this codebase for why the test suite exists.

BUG 1 — UNDEFINED BEHAVIOR IN READ_SAMPLE

Reading the top byte of a 32-bit sample did `in[off+3] << 24` — the byte is promoted to a signed `int` first, and for any byte value ≥ 128 , shifting it left by 24 overflows a signed 32-bit int, which C defines as undefined behavior. Confirmed directly by UndefinedBehaviorSanitizer: "left shift of 203 by 24 places cannot be represented in type 'int'." Fixed by casting to `uint32_t` *before* shifting, which makes the shift happen in well-defined unsigned arithmetic.

BUG 2 — THE UNMASKED DELTA (THE IMPORTANT ONE)

`sample - prev` was computed as plain 32-bit unsigned arithmetic regardless of the sample's actual width. For an 8-bit sample, `106 - 247` needs to wrap at 8 bits (→ 115), but the code produced the raw 32-bit wraparound value instead — a huge number representing -141 — which zig-zag mapping then turned into 281, too large to fit in the 8-bit field it gets written into. The write silently truncated it, corrupting every value after it. Invisible for 32-bit samples only because a `uint32_t`'s native wraparound happens to already match a 32-bit width — there was never an explicit mask. Caught by a round-trip test on random 8-bit data (a smooth ramp never triggers a negative wrap, which is exactly why that test alone wasn't enough).

BUG 3 — THE MAILBOX RACE

The dispatcher-to-worker mailbox originally had no backpressure: if `data` sent a second chunk before the worker had consumed the first, the second silently overwrote it. Fixed by making the handoff a proper synchronous rendezvous — the dispatcher now blocks briefly if the worker hasn't caught up, instead of clobbering unread data.

BUG 4 — DATA'S READ LOOP HAD NO BACKPRESSURE EITHER

Once bug 3 was fixed, the dispatcher (correctly) got slower under load — which then exposed a second, pre-existing gap: `data`'s `filesystem_stream_file` blasts every chunk in a tight loop with no throttling and never checked whether `IPC_send` actually succeeded. A fast reader could outrun `obc_ipc`'s connection backlog, and `IPC_send` is deliberately fail-fast (it does not retry — see its own comment in `obc_ipc.c`), so an overflowed chunk was just silently dropped — including, once, the very last chunk, which left the reader waiting forever for one that would never come. Fixed by having `filesystem_stream_file` retry a failed send a bounded number of times before giving up.

BUG 5 — A SAME-SIZE PROTOCOL COLLISION

`compute_cancel_request_t` (one `uint32_t`) and `data_write_ack_t` (one `int`) are both exactly 4 bytes — the dispatcher's original size-based message routing couldn't tell them apart, so a write-ack from `data` could be misinterpreted as a cancel request, and the worker would then hang forever waiting for an ack that had already arrived and been misrouted. Fixed by routing on the sender's role first (anything from `ROLE_DATA` is a reply for the worker; anything else is a request) rather than relying on size alone.

None of these were found by review. All five were found by writing code that actually exercises the behavior — a UBSan-instrumented round-trip fuzz test for 1–3, and directly running the real processes together under load for 3–5.

Part 4 — Protocol Reference

Two transports — do not confuse them

	Internal IPC	External CSP
Header	<code>obc_ipc.h</code>	<code>shared/csp/csp_commands.h</code>
Addressing	By role (<code>OBC_Roles_t</code> : <code>ROLE_MISSION</code> , <code>ROLE_DATA</code> , ...)	By CSP node address + port (<code>ADCS_ADDRESS</code> , <code>ADCS_CMD_PORT</code> , ...)
Used between	The 7 OBC processes, on one Linux box	OBC ↔ ADCS / EPS / etc., over the (simulated) I2C bus
Message cap	256 bytes (<code>MAX_IPC_PAYLOAD</code>)	Whatever CSP/the bus allows
Only used by	Every role, for everything internal	<code>commands</code> exclusively — no other process touches this

`obc_data_protocol.h`

```
data_read_request_t { char path[64]; }
data_read_reply_t   { int status; uint32_t offset; uint16_t length; uint8_t is_last;
uint8_t payload[200]; }
data_write_chunk_t  { char path[64]; uint32_t offset; uint16_t length; uint8_t
is_last; uint8_t payload[128]; }
data_write_ack_t    { int status; }
```

Write chunks carry a smaller payload (128 vs. 200 bytes) than read replies — they also carry their own `path` , since each write is a fully self-contained message with no session state kept in `data` between chunks (`offset==0` truncates/creates, `offset>0` appends), and that extra field has to fit under the same 256-byte cap.

`obc_compute_protocol.h`

```
compute_compress_request_t { uint32_t job_id; char in_path[64]; char out_path[64];
uint8_t sample_width; }
compute_cancel_request_t   { uint32_t job_id; }
compute_result_t           { uint32_t job_id; compute_status_t status; uint32_t
output_size; }

compute_status_t: OK=0, FAILED=1, CANCELLED=2, BUSY=3
```


Part 5 — Running and Testing the System

Build

```
cmake -S . -B build -DHW_MODE=OFF -DBUILD_TESTS=ON
cmake --build build
```

Run the whole OBC + ADCS

```
# terminal 1
./build/bin/obc_supervisor
# terminal 2
./build/apps/adcs/adcs_sim
```

`obc_supervisor` alone brings up all 6 other real roles. With both sides up, `mission` runs its scripted timeline on its own — watch for `[COMPUTE] job 1: done (... -> ... bytes)` when the compression step fires.

Run the automated test suite

```
ctest --test-dir build --output-on-failure
```

Test	What it proves
<code>comms_bus_test</code>	Two CSP nodes, addressed frames flow both directions over the real transport.
<code>comms_bus_addressing_test</code>	Three nodes — a frame addressed to one is <i>not</i> delivered to another.
<code>position_command_test</code>	Full-stack happy path against real ADCS, repeated cycles.
<code>command_ack_test</code>	ACK/NACK edge cases: bad input doesn't wedge the handler.
<code>full_constellation_test</code>	The real 7-process constellation, brought up exactly like a real launch, checked end to end.
<code>rice_codec_test</code>	Direct round-trip correctness of the compression codec, no processes involved.

<code>compute_async_test</code>	BUSY rejection and cancellation, against real running <code>data / compute</code> processes.
---------------------------------	--

All 7 pass on a clean build as of this document. `full_constellation_test` runs the real ~90-minute ascent wait and ~5-minute time-sync interval in seconds via `MISSION_ASCENT_WAIT_SEC / TIME_SYNC_INTERVAL_SEC` env-var overrides; `compute_async_test` similarly widens its job's runtime via `COMPUTE_CHUNK_DELAY_MS` so its timing-sensitive checks land reliably. None of these overrides change production behavior unless explicitly set.

Part 6 — Known Gaps

- **Real I2C HAL, real E22 radio driver, real camera driver** — all intentionally mock/stub today; the E22 radio is being built by a teammate and deliberately not touched here.
- **compute's codec is a subset of CCSDS 121.0** (no second-extension option, no reference-sample handling) — the options built cover what this project needs.
- **True image-aware compression (CCSDS 122.0)** is out of scope — a separate, much larger piece of work, and not worth building against today's mock text placeholder photo.
- **compute's single job slot** rejects a second concurrent request rather than queuing — right-sized for one photo per flight.
- **A shut-down board has no way back** except external intervention — needs EPS power-cycling, which doesn't exist yet.
- **ADCS's watchdog is POSIX-only** — real hardware needs an actual watchdog peripheral, part-specific and not yet built.
- **CMD_POINT_TO_SUN**, **CMD_SHUTDOWN**, **CMD_TIME_SYNC** are ACK+log placeholders on the ADCS side — real behavior needs hardware this project doesn't have yet.

See the README's "Known gaps" section for the complete, up-to-date list.

Part 7 — Glossary

CSP	CubeSat Space Protocol — the external wire protocol between boards (OBC, ADCS, etc.).
IPC	This project's own internal, role-addressed transport between the OBC's 7 processes. Not CSP.
CCSDS	Consultative Committee for Space Data Systems — the body that standardizes real spacecraft data formats, including 121.0 (lossless compression) and 122.0 (image compression).
Rice coding	A special case of Golomb coding, extremely cheap to compute, near-optimal for data whose values cluster around zero with roughly geometric probability.
Zig-zag mapping	A bijection from signed integers to non-negative integers that preserves "small magnitude" as "small value," needed because Rice coding only operates on non-negative numbers.
Delta / predictor	Replacing each sample with its difference from the previous one, so a correlated signal becomes a stream of small residuals.
Verbatim (block)	Storing a block's raw values uncompressed — the fallback used when entropy coding would do worse than nothing.
UBSan / ASan	UndefinedBehaviorSanitizer / AddressSanitizer — compiler-inserted runtime checks that catch memory and undefined-behavior bugs a normal build silently allows.
Heartbeat	A periodic no-payload IPC ping every long-lived role sends to <code>supervisor</code> so it isn't mistaken for a hung process.